

技术综述教学笔记

Agent 技术报告串讲: Kimi K2、ChatGPT Agent、Qwen3-Coder 与 Manus

Zhang Xiaojun 对话郑博元

基于 Zhang Xiaojun Podcast 公开视频、GPU 转写稿、官方技术报告/博文与自制概念图整理

2025-07-30



视频标题: 110. 逐段讲解 Kimi K2 报告并对照 ChatGPT Agent、Qwen3-Coder 等: “系统工程的力量”

视频频道: Zhang Xiaojun Podcast

视频时长: 02:20:45

视频链接: <https://www.youtube.com/watch?v=8dKBH4x0D9o>

素材说明: YouTube 无可读字幕; 正文基于 faster-whisper large-v3 CUDA 转写、视频章节、YouTube 描述、Kimi K2 技术报告、Qwen3-Coder 博文、Manus 上下文工程博文和自制教学图整理。固定封面视频不在正文重复使用。

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 导读：为什么这几份报告都指向 Agent	3
1.1 本章小结	4
2 Agent 定义与分类：感知 + 行动	4
2.1 四类 Agent	5
2.2 本章小结	6
3 两条技术路线：In-Context 与 End-to-End	7
3.1 Agent Training 三件套	7
3.2 本章小结	8
4 Kimi K2: Open Agentic Intelligence	9
4.1 MuonClip 与 QK-Clip	9
4.2 本章小结	10
5 ChatGPT Agent: 连接研究与实践	10
5.1 安全：新能力带来新风险	11
5.2 本章小结	12
6 Qwen3-Coder: Agentic Coding in the World	12
6.1 Code RL 与 Long-horizon RL	13
6.2 本章小结	14
7 Manus: 上下文工程是生产 Agent 的核心	14
7.1 KV cache 与工具遮蔽	14
7.2 本章小结	15
8 新范式：Environment + Task-Reward	15
8.1 Rubrics as reward 与 self-improvement	16
8.2 Family of Agents	17
8.3 本章小结	17
9 总结与延伸	18
9.1 关键 takeaways	18
9.2 开放问题	18

9.3 拓展阅读	19
----------------	----

1 导读：为什么这几份报告都指向 Agent

本节先建立整期的学习目标。这期节目读了几份近期最值得细读的 Agent 技术材料: Kimi K2 技术报告、ChatGPT Agent 发布文、Qwen3-Coder 技术博文，以及 Manus 的上下文工程经验。它们看似来自不同公司和产品，但共同指向一个主题：大模型正在从“回答问题”走向“在环境中完成任务”。

Agent 的关键不只是调用工具，而是完整闭环：感知环境、决策、行动、读取反馈、修正轨迹。这个闭环会牵出一整套工程问题：合成数据、轨迹生成、强化学习、可验证奖励、安全沙盒、上下文工程、KV cache、工具选择、文件系统记忆和多 Agent 组织。



图 1: 四份报告对照: Kimi、ChatGPT Agent、Qwen3-Coder、Manus 分别强调不同层。自制概念图，依据视频结构和官方材料整理。

读图：四份材料各看一层

Kimi K2 看开放模型和 agentic training，ChatGPT Agent 看统一产品系统，Qwen3-Coder 看代码环境里的 RL 和工具使用，Manus 看生产 Agent 的上下文工程。合起来就是 Agent 从模型、训练、产品到工程部署的全链条。

本期核心命题

Agent 的进步不是单一模型能力提升，而是系统工程：模型、环境、任务、工具、奖励、安全和上下文共同优化，才能把智能从输出文本推进到完成真实任务。

四份材料的学习顺序

材料	先看什么	教学价值
Kimi K2	MoE、MuonClip、agentic data、joint RL	看开放模型如何被训练成 Agentic。
ChatGPT Agent	Operator + Deep Research + 工具计算机	看 Agent 如何变成用户产品。
Qwen3-Coder	Code RL、long-horizon RL、Qwen Code	看代码环境为什么适合 Agent RL。
Manus	KV cache、工具遮蔽、文件系统、错误保留	看生产 Agent 如何靠上下文工程稳定运行。

关键数字速览

材料	关键数字/事实	教学含义
Kimi K2	约 1T 总参数、32B 激活参数、15.5T tokens	开放 MoE 模型也进入超大规模系统训练。
Kimi K2	SWE-Bench Verified 65.8、Tau2-Bench 66.1	Agentic benchmark 成为模型报告核心指标。
Qwen3-Coder	480B MoE、35B active、256K/1M context	代码 Agent 需要超长上下文和仓库级理解。
Qwen3-Coder	20,000 并行环境用于 long-horizon RL	Agent RL 的瓶颈是环境规模化。
Manus	输入/输出 token 比例约 100:1	KV cache 对成本和延迟极其关键。

1.1 本章小结

EP110 是 Agent 技术报告课。它把 EP115 的 Agent 理论、EP113 的 K2 模型公司视角、EP116 的企业级 Agentic Model 连接起来，展示“系统工程的力量”。

2 Agent 定义与分类：感知 + 行动

导读说明四份材料都指向 Agent，本章先把概念底座搭稳，回答一个具体问题：什么系统才算 Agent，而什么只是聊天机器人。视频描述给出的简洁定义是：Agent 是能够与环境进行交互的智能系统，具备感知能力和行动能力。感知包括观察环境、读取反馈、解析上下文；行动包括调用工具、生成输出、控制界面、修改变量。最小循环是“观察 → 决策 → 行动”。

Agent 最小定义

感知环境、决策、行动，再读取反馈



读图：从左到右看依赖关系；正文负责展开细节，图只保留骨架。

图 2: Agent 最小定义：感知环境、决策、行动，再读取反馈。自制概念图，依据 00:02:00–00:14:50 对谈内容整理。

读图：Agent 和 Chatbot 的差别

Chatbot 主要响应输入；Agent 要在环境中改变状态。只要系统开始调用工具、读反馈、调整计划，就从对话系统走向智能体系统。

2.1 四类 Agent

有了最小定义之后，本节按环境和产品形态分类。节目把 Agent 分成 Coding Agent、Search Agent、Tool-use Agent 和 Computer-use Agent。这个分类不是严格学术分类，而是产品和环境分类：代码、搜索、工具、电脑界面，对应不同 affordance 和验证方式。



图 3: Agent 类型地图: Coding、Search、Tool-use、Computer-use 对应不同环境。自制概念图, 依据 00:02:00-00:14:50 对话内容整理。

术语消化: Agent 类型		
类型	代表任务	难点
Coding Agent	代码补全、重构、调试、PR 修改	代码库上下文、测试、长程修改。
Search Agent	检索、汇总、调研报告	信息源可信、引用、去重和 synthesis。
Tool-use Agent	调用 API、函数、数据库和业务工具	工具选择、参数生成、错误恢复。
Computer-use Agent	操作浏览器、GUI、跨应用流程	视觉 grounding、状态追踪、权限安全。

2.2 本章小结

Agent 的定义可以很简单, 但落地非常复杂。不同 Agent 类型的本质差别, 在于它们面对的环境、工具和反馈不同。

3 两条技术路线：In-Context 与 End-to-End

上一章定义了 Agent 和环境类型，本章讨论怎样把 Agent 做出来。节目把路线分为 In-Context Learning 和 End-to-End Training。In-context 路线依赖强预训练模型、prompt、工具协议和上下文工程，迭代快、灵活性高；End-to-end 路线把行为写进模型权重，推理更稳定，但训练成本高、环境构建复杂。

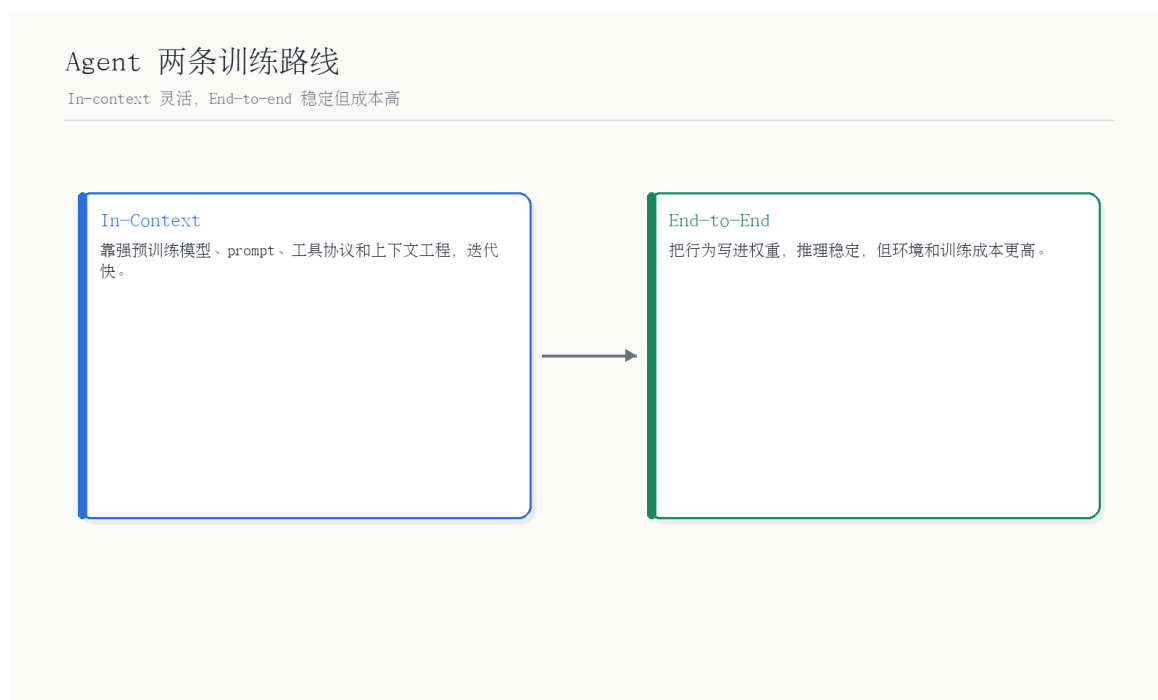


图 4: Agent 两条训练路线: In-context 灵活, End-to-end 稳定但成本高。自制概念图, 依据 00:14:50–00:30:57 对谈内容整理。

读图：两条路线不是非此即彼

很多真实系统会混合两者：用 in-context 快速搭产品，用合成轨迹和 RL 把关键行为训练进模型，再用上下文工程处理生产细节。

3.1 Agent Training 三件套

技术路线最终都要落到训练和部署，本节拆 Agent training 的三件套。Agent training 的关键环节包括数据合成、强化学习和安全。数据合成生成高质量 trajectory；强化学习依赖清晰 task 和 verifiable reward；安全则需要 sandbox、行为约束和 human-in-the-loop。

Agent Training 三件套

合成轨迹、强化学习和安全约束共同构成训练管线



读图：从左到右看依赖关系；正文负责展开细节，图只保留骨架。

图 5: Agent Training 三件套: 合成轨迹、强化学习和安全约束共同构成训练管线。自制概念图, 依据 00:28:29-00:30:57 对话内容整理。

Agent 数据的范式变化

传统标注数据常是 input-output; Agent 数据更像 environment + task + action trajectory + reward。模型学的不只是答案，而是如何在环境中行动。

训练环节对照

环节	产物	主要风险
Data Synthesis	高质量行动轨迹与工具调用示范	轨迹不真实，模型学会演戏。
RL	从环境反馈中优化策略	reward 设计错误或被 hack。
Safety	沙盒、权限、人工确认、拒绝策略	太松危险，太紧无法完成任务。
Evaluation	任务成功率、成本、恢复能力	benchmark 与真实任务不一致。

3.2 本章小结

Agent training 的核心是把任务放进环境，让模型通过轨迹、奖励和安全约束学习行动。它不是单纯 SFT，也不是单纯 prompt engineering。

4 Kimi K2: Open Agentic Intelligence

前两章讲 Agent 的通用框架，本章进入第一份技术报告，核心问题是：开放模型如何为 Agentic 能力做系统训练，并接近闭源前沿。Kimi K2 是 1T 参数级 MoE 模型，激活参数约 32B。报告提出 MuonClip 优化器，用 QK-Clip 处理 Muon 训练不稳定，同时保留 token efficiency；K2 在 15.5T tokens 上预训练，后训练包含大规模 agentic data synthesis 和 joint RL。



图 6: Kimi K2 Agentic Pipeline: MuonClip、15.5T tokens、agentic data synthesis 与 joint RL。自制概念图，依据 Kimi K2 技术报告和 00:30:57–00:43:50 对谈内容整理。

读图：Kimi K2 的重点不是单一 benchmark

K2 把 MoE 底座、稳定优化器、大规模数据、工具轨迹合成和 RL 放在一起。它的 agentic 能力来自一整条训练管线，而不是只靠后处理。

4.1 MuonClip 与 QK-Clip

读懂 K2，不能只看 agentic benchmark，也要看它如何稳定训练。本节解释 MuonClip 与 QK-Clip。Muon 有 token efficiency 优势，但大规模训练可能不稳定；QK-Clip 通过约束 attention logits 来控制训练发散风险。报告强调，MuonClip 使 K2 能在 15.5T tokens 训练中保持稳定。

术语消化: Kimi K2 训练组件

组件	作用	风险/意义
MoE	增加总参数但控制激活计算	路由、专家负载和训练稳定性复杂。
MuonClip	提升 token efficiency 并稳定训练	需要处理 attention logits 爆炸。
QK-Clip	约束 query/key 投影导致的 logits	属于最小干预式稳定机制。
Agentic data synthesis	生成工具使用轨迹和任务数据	质量过滤和奖励设计很关键。
Joint RL	在真实/合成环境中提升能力	环境和 reward 成本高。

4.2 本章小结

Kimi K2 的教学价值在于，它把开放模型、MoE、优化器稳定性、合成工具轨迹和 RL 组合成一个 agentic training pipeline。

5 ChatGPT Agent: 连接研究与实践

Kimi K2 展示模型训练管线，本章转到产品系统，核心问题是：训练出来的 Agent 如何进入用户 workflow，并承担真实操作风险。第二份材料是 ChatGPT Agent 发布文。OpenAI 将 Operator、Deep Research 和 ChatGPT 对话能力整合成统一的智能体系统。它通过自己的虚拟计算机、文本浏览器、可视化浏览器、终端和 API 访问来完成任务，同时保留用户确认、接管和中断能力。



图 7: ChatGPT Agent 统一系统: Operator + Deep Research + ChatGPT + 工具计算机。自制概念图，依据 OpenAI 官方页面和 00:43:50–01:53:38 对谈内容整理。

读图：产品系统比模型多很多层

ChatGPT Agent 不只是模型，它包含浏览、搜索、代码执行、文件处理、连接器、用户确认和安全策略。这些层决定它能不能处理真实工作流。

ChatGPT Agent 工具栈

工具/层	作用	风险控制
视觉浏览器	与人类网页交互，点击、筛选、填写	用户可接管浏览器。
文本浏览器	高效读取大量网页文本	需要来源过滤和引用。
Terminal	运行代码、处理文件和数据	高风险操作需限制。
Connectors/API	连接 Gmail、GitHub、日历等	权限和隐私控制。
User confirmation	执行重要操作前确认	降低误操作后果。

5.1 安全：新能力带来新风险

OpenAI 页面强调，新型 Agent 能在网页上执行操作，因此带来提示注入、误操作、隐私和高风险任务问题。控制措施包括重要操作确认、接管模式、监控模式、拒绝高风险任务、限制数据访问和安全浏览器接管。

Agent 安全的核心变化

聊天模型说错话，后果通常还在文本层；Agent 做错动作，可能影响账户、文件、邮件、购买、代码和真实业务。因此安全从内容审核扩展到权限、工具、确认和审计。

安全风险分层

风险	例子	防护思路
提示注入	网页隐藏指令诱导 Agent 泄露数据	检测、隔离、重要动作确认。
误操作	误发邮件、误购买、误删文件	人类确认、接管、权限分级。
隐私泄露	连接器或登录网站数据被读取	最小权限、会话清理、禁用无关连接器。
高风险任务	金融、生物、危险操作	拒绝策略、监控模式、专门防护。

5.2 本章小结

ChatGPT Agent 的重点是统一产品系统：它把研究能力、网页操作、工具执行和用户控制放进同一 workflow，展示了 Agent 从实验到产品的路径。

6 Qwen3-Coder: Agentic Coding in the World

前一章看通用产品 Agent，本章看代码环境里的 Agent，核心问题是：为什么 coding 是最适合 Agent 训练的高价值环境之一，以及开源模型如何在这个环境里追赶。第三份材料是 Qwen3-Coder。官方博文介绍 Qwen3-Coder-480B-A35B-Instruct: 480B MoE、35B active，原生 256K context，通过 YaRN 可扩到 1M。它面向 agentic coding，强调 code RL、long-horizon RL、20,000 并行环境，以及 Qwen Code 工具链。

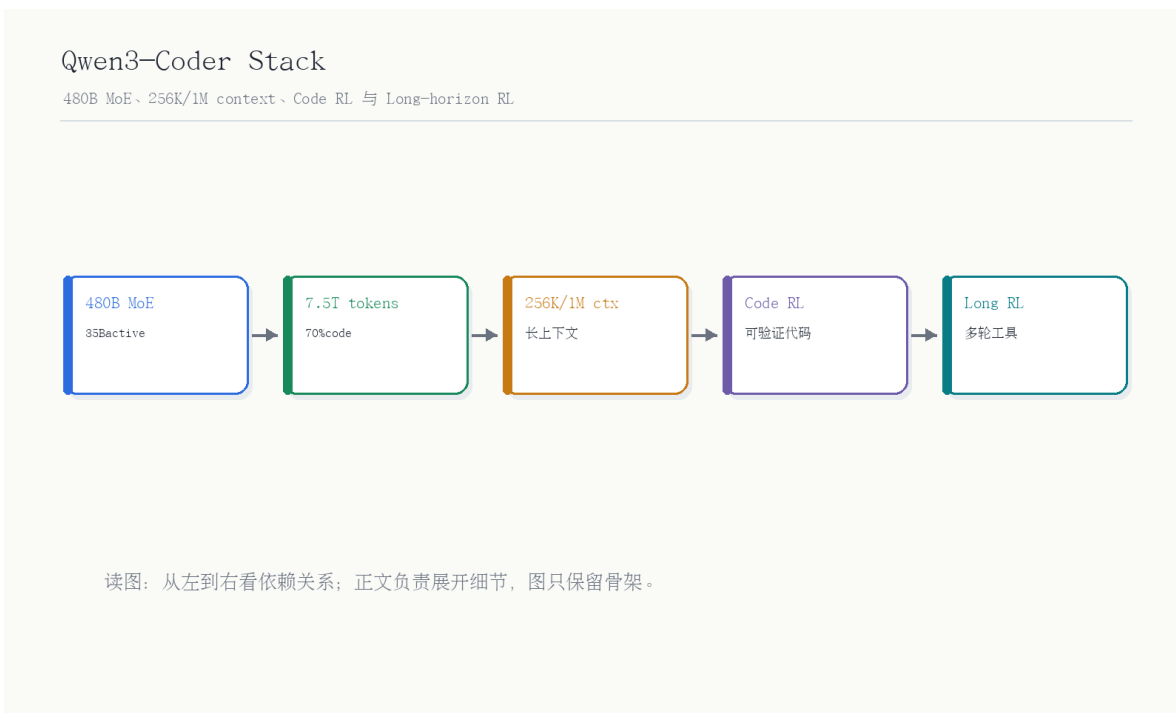


图 8: Qwen3-Coder Stack: 480B MoE、256K/1M context、Code RL 与 Long-horizon RL。自制概念图，依据 Qwen 官方博文和 01:53:38–01:59:04 对谈内容整理。

读图：代码是最适合 Agent RL 的环境之一

代码任务 hard to solve, easy to verify: 解决很难，但测试和执行反馈相对明确。这让代码成为大规模 RL 和 long-horizon interaction 的好环境。

6.1 Code RL 与 Long-horizon RL

上一节给出 Qwen3-Coder 的模型栈，本节解释它为什么强调 RL。Qwen3-Coder 博文强调，代码任务天然适合 execution-driven RL。真实软件工程任务需要多轮交互：规划、使用工具、接收反馈、修正决策。Qwen3-Coder 引入 long-horizon RL，并构建可并行运行的大规模环境。

为什么 coding 是 Agent 的高地

代码环境有清晰动作、可执行反馈、测试验证和高价值任务。相比很多开放世界任务，coding 更容易定义 reward，也更容易规模化训练和评测。

Code RL 为什么“难解但易验”

很多代码任务的解决过程很难，需要理解代码库、规划修改和调试；但结果可以通过测试、编译、lint、benchmark 或用户脚本验证。这种结构非常适合强化学习，因为 reward 可以相对自动化。

6.2 本章小结

Qwen3-Coder 展示了开源模型在 agentic coding 上的系统路线：长上下文、代码数据、执行驱动 RL、长程工具交互和 CLI 工具链。

7 Manus: 上下文工程是生产 Agent 的核心

Kimi、ChatGPT Agent、Qwen 分别展示模型、产品和代码 RL，本章看生产 Agent 最容易被低估的一层：上下文工程。Manus 选择基于前沿模型的 in-context 能力构建 Agent，而不是从头训练端到端模型。它强调上下文工程是一门实验科学，生产 Agent 的关键不只是 prompt，而是 KV cache、工具管理、文件系统记忆、todo 复述、保留错误和避免少样本陷阱。

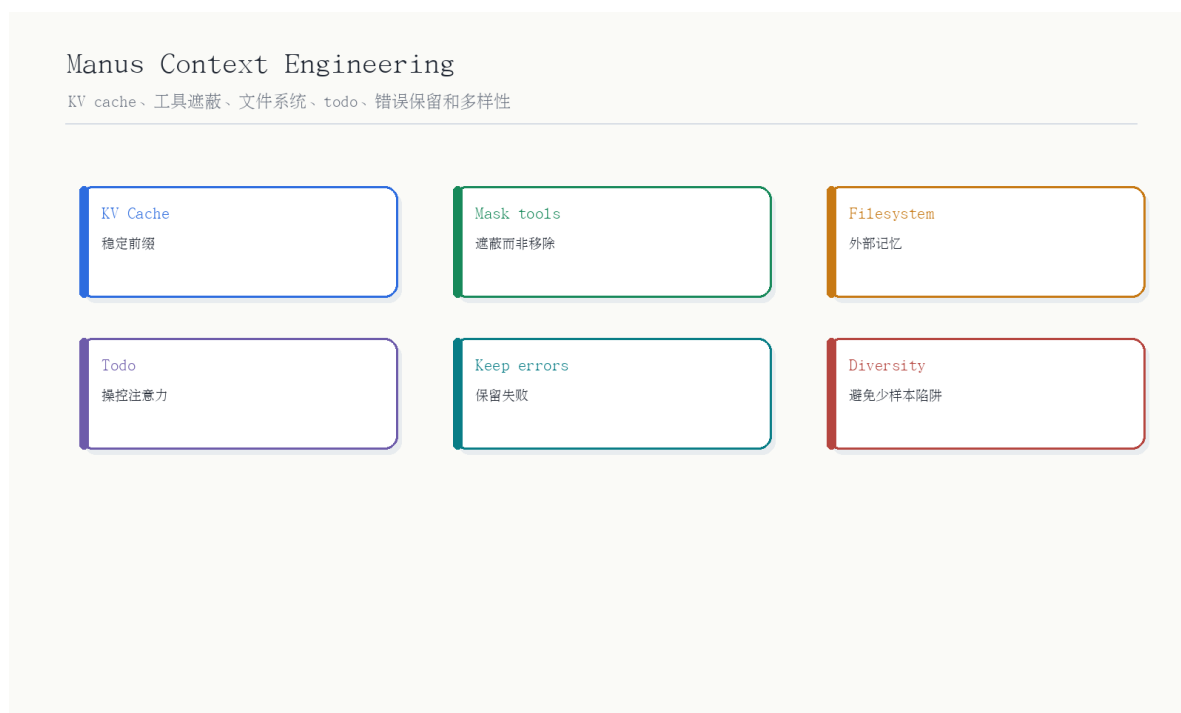


图 9: Manus Context Engineering: KV cache、工具遮蔽、文件系统、todo、错误保留和多样性。自制概念图，依据 Manus 官方博文和 01:59:04-02:06:06 对谈内容整理。

读图：上下文工程是运行时工程

训练让模型有能力，上下文工程让 Agent 在生产中稳定、便宜、可恢复地使用能力。它处理的是每一轮工具调用、上下文增长和失败恢复。

7.1 KV cache 与工具遮蔽

上一节说上下文工程是运行时工程，本节拆两个最实用的原则：KV cache 和工具遮蔽。Manus 认为 KV-cache 命中率是生产 Agent 最重要指标之一，因为 Agent 的输入输出比例高度偏向输入，缓存直接影响延迟和成本。它还主张“遮蔽，而非移除”工具：动态增删工具会破坏缓存并混淆模型；更好的做法是保留工具定义，用状态机或 logits mask 控制可选工具。

术语消化：上下文工程关键点

技术	作用	错误做法
KV cache	降低长前缀重复推理成本	每轮改变系统 prompt 或工具定义。
Tool masking	控制当前可用动作	动态删除工具导致上下文不一致。
Filesystem as context	用文件系统做外部长期记忆	把所有内容塞进上下文窗口。
Todo.md	通过复述目标操控注意力	长任务中不刷新目标。
Keep errors	保留失败证据帮助恢复	清理错误导致重复犯错。

生产 Agent 检查清单

一个生产 Agent 至少要回答六个问题：上下文是否可缓存？工具空间是否可控？长期记忆在哪里？失败是否保留？目标是否会被复述到近期上下文？重复示例是否会让模型陷入模式？Manus 的经验几乎都围绕这些问题展开。

生产 Agent 系统 checklist

检查项	问题	不做的后果
缓存	前缀是否稳定、上下文是否 append-only?	成本和延迟暴涨。
工具	工具定义是否稳定、动作是否可约束?	选错工具、缓存失效、动作幻觉。
记忆	长期状态是否外部化到文件或数据库?	上下文爆炸或信息丢失。
错误	失败轨迹是否保留给模型学习?	重复同一错误。
安全	高风险动作是否确认和审计?	Agent 误操作造成真实损害。

7.2 本章小结

Manus 提醒我们，Agent 的生产问题常常不是模型不够强，而是上下文、工具、记忆和失败恢复没有设计好。

8 新范式：Environment + Task-Reward

本章收束到节目里的展望：新的数据核心可能从 input-output 标注，转向 environment 和 task-reward 构造。也就是说，未来训练 Agent 的核心不只是收集答案，而是构造可交互环境、定义任务、设计可验证 reward，让模型从经验中 self-improve。

Agent 新范式

数据核心从 input-output 转向 environment + task-reward



读图：从左到右看依赖关系；正文负责展开细节，图只保留骨架。

图 10: Agent 新范式：数据核心从 input-output 转向 environment + task-reward。自制概念图，依据 02:06:06–02:15:20 对话内容整理。

读图：从答案数据到经验数据

Input-output 数据教模型“看到输入给出答案”；environment + task-reward 教模型“在环境中行动并从反馈中改进”。这就是 Agent 与普通聊天模型的数据范式差异。

8.1 Rubrics as reward 与 self-improvement

上一节讲新数据范式，本节聚焦它的核心难题：reward 从哪里来。节目提到 rubrics as reward，即用评分标准作为奖励机制。它的意义是：很多现实任务没有简单对错，但可以有分层评分标准。Agent 能不能自我提升，取决于它能否找到或构造可验证 reward，并有效利用交互经验。

Self-improve 的前提

Agent 自我提升不是让模型无限自嗨。它需要可验证任务、可靠环境、可审计轨迹和防止 reward hacking 的安全机制。

新范式对照表

范式	数据形态	主要瓶颈
Input-output	输入和标准答案	标注成本和泛化边界。
Trajectory	观察、动作、工具调用、结果	轨迹质量和错误恢复。
Environment	可交互任务世界	环境构建成本和覆盖度。
Task-reward	任务定义和可验证反馈	reward 设计和安全。
Experience reuse	利用历史经验自我提升	记忆、去噪和防止自嗨。

8.2 Family of Agents

reward 和 self-improvement 讨论的是单体系统如何学习，本节转向多 Agent 组织。结尾提到 Agent 像“拓展的大脑”，背后有一个军团。这个比喻说明未来 Agent 可能不是一个单体，而是一组专门化智能体：coding、search、browser、memory、safety 等共同协作。

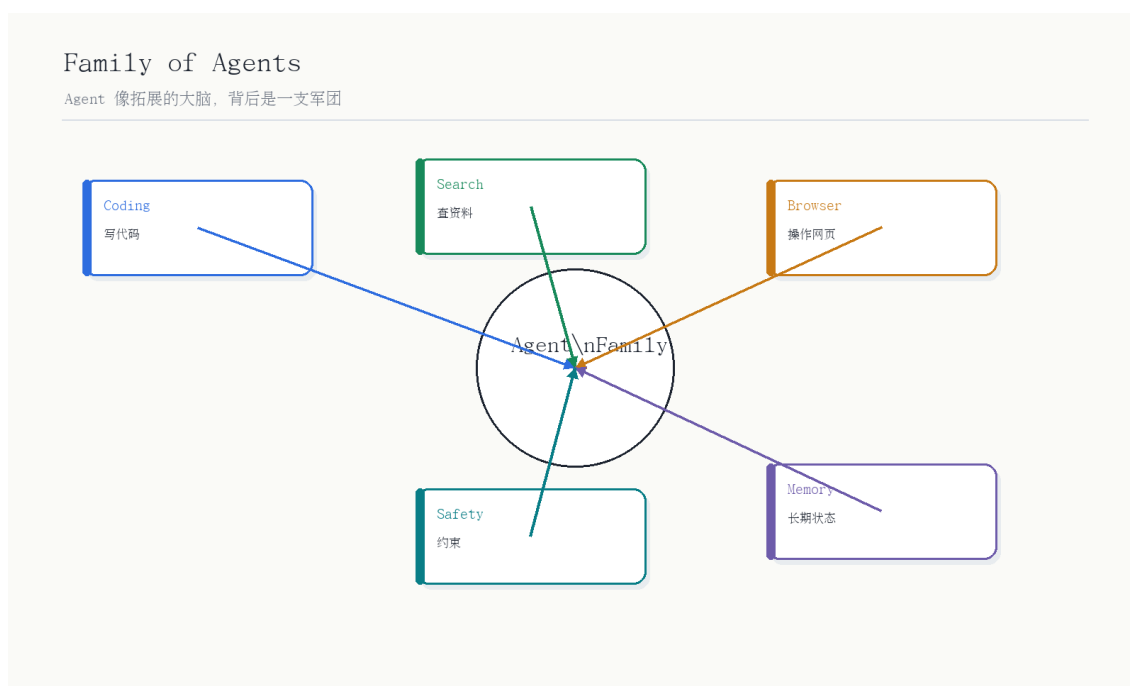


图 11: Family of Agents: Agent 像拓展的大脑，背后是一支军团。自制概念图，依据 02:15:20–02:16:41 对谈内容整理。

8.3 本章小结

Agent 的新范式是把数据、环境、奖励和经验循环合在一起。真正的系统工程，是让这些循环可规模化、可验证、可安全部署。

9 总结与延伸

本节把 EP110 压缩成几个结论。第一, Agent 的最小定义是感知和行动, 而不是聊天。第二, Kimi K2、ChatGPT Agent、Qwen3-Coder 和 Manus 分别展示了 Agent 栈的模型、产品、代码/RL 和上下文工程层。第三, Agent training 的核心从 input-output 走向 trajectory、environment、task-reward 和 safety。第四, 生产 Agent 的关键是系统工程: KV cache、工具遮蔽、文件系统记忆、错误恢复和用户控制。

把 EP110 放进张小珺 AI 队列

EP115 给出 Agent 下半场理论, EP113 讲 Kimi K2 的模型公司视角, EP116 讲企业级 Agentic Model; EP110 则把几份最新技术报告合在一起, 展示 Agent 从论文、模型、产品到工程落地的完整链条。

与前后几集的关系

节目	主题	与 EP110 的连接
EP115	Agent 下半场理论	给出 reward、environment、interface 的理论框架。
EP113	Kimi K2 和 Agentic LLM	给出模型公司如何理解 K2 和开源生态。
EP116	企业级 Agentic Model	给出 Agent 在 ToB 私有数据中的部署视角。
EP139	Agent 技术史	给出 Agent 从早期系统到 LLM Agent 的历史脉络。

9.1 关键 takeaways

前面章节已经分别从定义、训练、产品、代码环境和上下文工程解释 Agent。本节把这些内容压缩成几条工程判断, 方便后续和 EP115、EP113、EP116 对照。

1. Agent = 感知 + 行动 + 反馈, 不是“会调用工具”的单一能力。
2. Kimi K2 的重点是开放 MoE 模型、MuonClip、合成工具轨迹和 joint RL。
3. ChatGPT Agent 的重点是统一产品系统和用户控制。
4. Qwen3-Coder 的重点是代码环境、Code RL、Long-horizon RL 和工具链。
5. Manus 的重点是上下文工程, 特别是 KV cache、工具遮蔽和文件系统记忆。

9.2 开放问题

这些问题是 Agent 从报告走向产品后仍然没有定论的部分。它们决定下一轮训练范式、产品架构和基础设施投入方向。

1. Agent 训练中，合成轨迹和真实环境反馈如何配比？
2. Rubrics as reward 能否支撑开放任务中的可靠 self-improvement？
3. In-context 与 end-to-end 两条路线最终会融合到什么形态？
4. 生产 Agent 的成本瓶颈会更主要来自模型调用、工具执行还是上下文长度？

9.3 拓展阅读

- 对 Agent 理论框架感兴趣，可对照 EP115 姚顺雨访谈。
- 对 Kimi K2 模型公司视角感兴趣，可对照 EP113 杨植麟访谈。
- 对企业 Agentic Model 感兴趣，可对照 EP116 吴明辉访谈。